

CONTENTS

Talk²Books

Phase 4 — Complete Build Guide & Interview Preparation

A beginner-friendly guide to building a fully local, multilingual AI-powered document assistant from scratch.

WHAT'S INSIDE

- + Complete tech stack explained simply --- no jargon
- + Phase 4 build journey, day by day
- + How the RAG pipeline actually works
- + 10 questions with ideal answers
- + 10 interview questions with full ideal answers
- + Glossary
- + Glossary of every technical term used

LOCAL

PRIVATE

MULTILINGUAL

AI-POWERED

SECTION 01

What is Talk2Books?

Imagine you have a pile of books, contracts, research papers, or notes. You want a specific answer from them — but you do not want to read everything. Talk2Books solves exactly this. It is a web application that lets you **chat with your documents** by asking questions in plain English (or Hindi, Punjabi, or Sanskrit) and getting precise answers — with the exact source passage highlighted.

The special part: **everything runs on your own computer**. No data is sent to OpenAI, Google, or any cloud service. Your documents stay completely private. The AI model runs locally, the database runs locally, and the web server runs locally.

"Think of Talk2Books as a very smart library assistant who has read all your books and can instantly find the right page when you ask a question."

What Problem Does It Solve?

Normal AI chatbots like ChatGPT answer from their training data — they have never seen your specific documents. If you ask "What does my contract say about termination?", ChatGPT will guess or make something up. This is called a **hallucination**.

Talk2Books fixes this by first searching your document for the most relevant passages, then giving those passages to the AI as context. The AI answers based on **real content from your document**, not guesswork. This approach is called **RAG** — Retrieval-Augmented Generation.

What is RAG? (Explained Simply)

RAG has three steps:

- **R — Retrieve:** When you ask a question, the system searches your documents for the most relevant passages.
- **A — Augment:** Those passages are added to your question to create an enriched prompt.
- **G — Generate:** The AI reads the enriched prompt and generates an accurate, grounded answer.

Without RAG: You ask AI about your contract → AI guesses → wrong answer.

With RAG: AI reads your actual contract → gives a real, cited answer.

Who Is This For?

Anyone who works with documents — students researching from textbooks, lawyers reviewing contracts, researchers reading papers, or professionals working with technical manuals. Talk2Books turns any document into an interactive assistant.

SECTION 02

The Technology Stack

Talk2Books is built with 9 key technologies. Each one has a specific job. Think of them like the different parts of a restaurant: the dining room (frontend), the kitchen (backend), the pantry (database), and the chef (AI).

React 18

The User Interface

What it is: A JavaScript framework from Meta for building interactive web UIs.

Why we use it: React powers everything you see and click: the three-panel layout (Library sidebar, Chat center, Sources panel), message bubbles, document list, and the animated thinking dots.

Where in TTB: frontend/src/app.jsx — the entire visual layer

Vite

Development Build Tool

What it is: A lightning-fast development server and bundler for JavaScript projects.

Why we use it: Vite serves the React app during development and bundles it for production. Critically, it also acts as a proxy — all /api/* requests from the browser are forwarded to the Python backend, solving the CORS problem completely without any configuration on the backend.

Where in TTB: frontend/vite.config.js — proxy config

Python + Quart

The Backend API Server

What it is: Quart is an async Python web framework (like Flask, but non-blocking).

Why we use it: Quart is the bridge between the React frontend and the AI pipeline. It exposes four REST endpoints: GET /api/health (server status), GET /api/documents (list ingested docs), POST /api/query (ask a question), and POST /api/upload (upload a new document).

Where in TTB: backend/app.py

Qdrant

The Vector Database

What it is: A database that stores text as mathematical vectors and finds similar content.

Why we use it: Qdrant is the "memory" of Talk2Books. Every document chunk is stored as a 768-number vector. When you ask a question, Qdrant finds the 3 most mathematically similar chunks. It also supports metadata filtering, which we use to search only within the selected document and the correct language.

Where in TTB: Runs locally on localhost:6333. Used in both ingest.py and rag_chain.py

Ollama + phi4-mini

The Local AI Model

What it is: Ollama is a tool for running open-source AI models locally. phi4-mini is a lightweight but capable language model from Microsoft.

Why we use it: After retrieving relevant chunks, the system sends a structured prompt to phi4-mini via Ollama. The model reads the passages and generates a grounded answer. It runs entirely offline — no internet, no API key, no subscription needed.

Where in TTB: Runs locally on localhost:11434. Called in backend/rag_chain.py

Sentence Transformers (all-mpnet-base-v2)

The Embedding Model

What it is: A HuggingFace model that converts text into 768-dimensional numerical vectors.

Why we use it: To search for "similar meaning" rather than exact words, we must convert text into numbers that computers can compare mathematically. all-mpnet-base-v2 converts both document chunks (at ingest time) and user questions (at query time) into the same 768-number format, enabling semantic similarity search.

Where in TTB: Used in backend/ingest.py (ingestion) and backend/rag_chain.py (queries)

LangChain

The AI Pipeline Coordinator

What it is: A Python library that connects AI components — loaders, splitters, embeddings, and vector stores — into clean pipelines.

Why we use it: LangChain provides ready-made document loaders (PyPDFLoader, Docx2txtLoader), the RecursiveCharacterTextSplitter for chunking, HuggingFaceEmbeddings for vectorisation, and the QdrantVectorStore for storage. Without LangChain, we would need to write all of this integration code ourselves.

Where in TTB: backend/ingest.py and backend/rag_chain.py

langdetect

Language Detection Library

What it is: A Python library that identifies which human language a text is written in.

Why we use it: Talk2Books supports English, Hindi, Punjabi, and Sanskrit. langdetect automatically identifies the language of both documents (at ingest) and questions (at query time). This drives the language filter in Qdrant, ensuring you get responses in the right language.

Where in TTB: backend/ingest.py and backend/rag_chain.py

react-markdown

Markdown Renderer for React

What it is: A React component that converts markdown syntax into formatted HTML.

Why we use it: phi4-mini often returns answers with **bold text**, bullet lists, and code blocks written in markdown format. Without react-markdown, these appear as raw asterisks and hyphens. With it, responses render as properly formatted, readable text.

Where in TTB: frontend/src/app.jsx — wrapped around AI message content

SECTION 03

Phase 4 — Day by Day

Phase 4 is the full-stack integration phase — connecting the Python backend built in Phases 1-3 to a working React frontend. Here is exactly what was built each day, in plain terms.

DAY 8

React Frontend Shell + GET /api/documents

The first task was creating the three-panel React shell (Library, Chat, Sources) and wiring the first real API connection. The GET /api/documents endpoint was added to app.py — it scrolls through all Qdrant points, extracts unique file_name values from metadata, and returns them as a list so the sidebar can display your documents.

- Three-panel layout (Library | Chat | Sources) built in app.jsx
- GET /api/documents queries Qdrant and deduplicates by file_name
- Frontend fetchDocs() calls the endpoint on page load
- Active document state tracks which document is selected

DAY 9

Fix the Source Data Mismatch

There was a silent bug: the backend returns two separate arrays — "sources" (just filename strings) and "chunks" (full objects with text, score, and metadata). The frontend's sendQuestion() was picking up "sources" first, so the right panel was getting strings instead of objects.

- Added normalizeChunk() to reshape chunk objects to the SourceCard format
- Fixed sendQuestion() to use data.chunks instead of data.sources
- Source cards now display text snippets, relevance scores, and progress bars

DAY 10

Wire the Language Selector

The EN/HI/PA/SA dropdown existed in the UI but was completely disconnected. This day wired it end-to-end: React state tracks the selection, it is sent in every API call, and the backend skips auto-detection when a language is explicitly provided.

- language useState added and bound to the select element
- language field included in the /api/query request body
- app.py extracts language and passes to ask()
- retrieve_chunks() uses provided language instead of detecting
- Backend logs show "manual" vs "auto-detected" language

DAY 11**Per-Document Filtering in the RAG Chain**

Before this day, every question searched the entire Qdrant collection regardless of which document was selected in the sidebar. This meant asking about "Contract.pdf" might return chunks from "Textbook.pdf" too.

- Frontend sends doc_id (the filename) in every /api/query request
- app.py extracts doc_id and passes it to ask()
- retrieve_chunks() builds a Qdrant FieldCondition on metadata.file_name
- Language filter and doc filter are combined into one Qdrant Filter object
- Fallback: if no results with language filter, retry with doc_id only

DAY 12**Markdown Rendering for AI Responses**

phi4-mini frequently returns responses formatted with markdown — **bold** words, - bullet lists, and ``code`` blocks. Without a renderer, these appear as raw asterisks and hyphens, looking broken.

- Installed react-markdown package
- AI message bubbles now wrapped in
- User messages remain plain text (no markdown needed)
- CSS added for p, ul, ol, li, code, and pre elements inside AI bubbles

DAY 13**Clickable Citation Badges**

The [1] [2] [3] number badges that appear in AI responses were decorative only — clicking them did nothing. This day wired them to highlight and scroll to the matching source card on the right panel.

- activeSource state tracks which badge was last clicked
- handleCitationClick() sets state and calls scrollIntoView
- Each SourceCard gets a unique id="source-card-N" for targeting
- Active source card highlights gold and active state resets on new query

DAY 14

Document Upload — Backend

The "Upload Document" button existed in the UI but was never connected. Day 14 built the backend half: a new endpoint that accepts a file, validates it, saves it to the `data/` folder, and ingests it into Qdrant.

- `POST /api/upload` added to `app.py`
- File type validation: `.pdf`, `.docx`, `.txt`, `.csv` only
- `ingest_file()` added to `ingest.py` for single-file pipeline
- Returns document metadata: `id`, `name`, `language`, `chunk count`
- Cleans up saved file if ingestion fails

SECTION 04

The Data Flow

When You Upload a Document

Load: The file is read using the correct loader. PDFs use PyPDFLoader, Word files use Docx2txtLoader, plain text uses TextLoader.

Normalize: Text is cleaned — extra whitespace collapsed, Unicode normalized to NFC format.

Detect Language: langdetect reads the first 500 characters and identifies the language (en, hi, pa, etc.). This is stored in metadata.

Chunk: The document is split into 500-character pieces with 50-character overlap using RecursiveCharacterTextSplitter.

Embed: Each chunk is converted to a 768-dimensional vector by all-mpnet-base-v2.

Store: Chunks + their vectors + their metadata are upserted into Qdrant collection "ttb_documents".

When You Ask a Question

Receive: The React frontend sends a POST /api/query with: question, language, and doc_id.

Embed Query: The question is converted to a 768-dimensional vector using the same all-mpnet-base-v2 model.

Filter Build: A Qdrant filter is built: must match metadata.file_name = doc_id AND metadata.language = lang.

Vector Search: Qdrant finds the 3 chunks whose vectors are mathematically closest (cosine similarity) to the question vector.

Prompt Build: The 3 chunks are formatted into a structured prompt with the question and clear instructions for the AI.

LLM Generation: The prompt is sent to phi4-mini via Ollama. The model reads the passages and generates a grounded answer.

Return: The API returns: answer (text), sources (filenames), language, and chunks (for the Sources panel).

Display: React renders the answer with react-markdown, populates the source cards, and shows citation badges.

Key insight: the embedding model is used TWICE — once when ingesting (to store chunks) and once when querying (to convert the question). Using the same model both times is critical. If you stored with Model A and searched with Model B, the vectors would be incompatible and results would be garbage.

SECTION 05

Interview Questions & Ideal Answers

These are the questions a technical interviewer is most likely to ask about this implementation. Each answer explains the reasoning, not just the facts.

QUESTION 01

Can you explain what RAG is and why you chose it over fine-tuning?

RAG stands for Retrieval-Augmented Generation. Instead of training the AI model on my specific documents (which is called fine-tuning and requires significant compute, time, and money), RAG keeps the model frozen and feeds it relevant document passages at query time.

The key advantage is **flexibility**. With fine-tuning, you retrain every time you add a document. With RAG, you just ingest the new document into Qdrant and it is immediately queryable. Fine-tuning also bakes knowledge into the model weights permanently — if the document changes, you retrain. RAG uses live retrieval, so it always reflects the current document.

For Talk2Books specifically, users upload different documents every session. RAG is the only practical choice.

QUESTION 02

How does vector similarity search work in your system?

Text cannot be compared mathematically in its raw form. We first convert it to a vector — a list of 768 floating-point numbers — using the all-mpnet-base-v2 sentence transformer model. This model was trained to place semantically similar sentences **close together** in this 768-dimensional space.

When a user asks "What are the project deliverables?", we convert that question to a vector. Qdrant then computes the **cosine similarity** between this question vector and every stored chunk vector. Cosine similarity measures the angle between two vectors — angle near zero means very similar meaning. The top 3 closest chunks are returned.

This is why it finds relevant content even when the exact words differ — "deliverables" and "output items" would have similar vectors because they appear in similar contexts in training data.

QUESTION 03

Why did you choose Qdrant over alternatives like Pinecone or ChromaDB?

The primary reason is that Qdrant runs completely locally with a single Docker command or binary, which aligns with the core requirement of Talk2Books: **zero cloud dependency, total privacy**.

Pinecone is cloud-only — it would require sending document data to an external server, which violates the privacy requirement. ChromaDB is a good local alternative, but Qdrant has more mature metadata filtering (which we rely on heavily for doc_id and language filters), better performance documentation, and a more production-ready architecture.

Qdrant's Python client also integrates cleanly with LangChain's QdrantVectorStore, making the integration straightforward.

QUESTION 04

Explain your chunking strategy. Why 500 characters with 50 overlap?

Chunking is the step where we split documents into pieces for storage. The chunk size is a trade-off between two competing needs. **Too small** (e.g. 100 chars) and each chunk loses context — you might get half a sentence with no meaning. **Too large** (e.g. 2000 chars) and the vector becomes a blur of many different ideas, making similarity search imprecise.

500 characters is approximately 2-4 sentences — enough to carry a complete thought while remaining focused on a single topic. The 50-character overlap ensures that important sentences at chunk boundaries are not lost — the end of one chunk repeats slightly at the start of the next.

We use `RecursiveCharacterTextSplitter` which tries to split at paragraph breaks first, then sentences, then words, and only as a last resort at arbitrary character positions. This preserves semantic boundaries.

QUESTION 05

How did you handle per-document filtering? Walk me through the implementation.

Before Day 11, every query searched the entire Qdrant collection regardless of which document was active. This is a correctness problem — answers could come from any document.

The fix works at three layers. In the **frontend**, the active document filename is sent as `doc_id` in every query body. In **app.py**, this is extracted and passed to the `ask()` function. In **rag_chain.py**, `retrieve_chunks()` builds a Qdrant Filter with a `FieldCondition` on `metadata.file_name` matching the `doc_id`.

The language filter and `doc_id` filter are combined into a single Filter object with a "must" list — both conditions must match. There is also a fallback: if the combined filter returns no results (perhaps the document is in English but the question is in Hindi), we retry with only the `doc_id` filter to ensure the user always gets an answer from the correct document.

QUESTION 06

How does the frontend communicate with the backend? How did you solve CORS?

The React frontend runs on port 3000 via Vite. The Quart backend runs on port 5000. Browsers enforce the **Same-Origin Policy** — they block requests between different ports by default, which would cause a CORS error.

The solution is Vite's built-in proxy configured in `vite.config.js`. All requests to `/api/*` from the browser go to port 3000 (same origin). Vite's dev server intercepts them and forwards them to `localhost:5000` server-side. The browser never makes a cross-origin request — so no CORS issue at all. This is the cleanest solution because it requires zero backend configuration.

In production, the static React build would be served from the same origin as the API, so CORS is not an issue there either. `quart_cors` is also installed as a backup for development scenarios.

QUESTION 07

What is the `normalizeChunk()` function and why was it needed?

This was a bug fix from Day 9. The backend's `/api/query` response contains two arrays: "sources" (an array of filename strings like `["file.pdf"]`) and "chunks" (an array of full objects with `text`, `score`, `source`, `language`, and `file_name` fields).

The JavaScript `??` (nullish coalescing) operator stops at the first truthy value. Since "sources" is always an array (even if it just contains strings), it was always truthy, so the code never reached "chunks". The `SourceCard` component received strings instead of objects and rendered nothing useful.

`normalizeChunk()` also handles a shape mismatch: the backend uses "text" for content, but `SourceCard` reads "content". It also wraps source and page info into a nested "metadata" object matching the component's expected structure. The fix was to explicitly target `data.chunks` and run each item through `normalizeChunk()` before storing in state.

QUESTION 08

Why did you choose `phi4-mini` as the language model?

The non-negotiable requirement for Talk2Books is complete local operation — no API keys, no internet connection, no data leaving the machine. This rules out GPT-4, Claude, and Gemini.

`phi4-mini` is Microsoft's lightweight model specifically designed for efficiency on consumer hardware. It runs on 4-8GB RAM, responds in acceptable time on a MacBook without a GPU, and produces sufficiently accurate answers for document Q&A; tasks (which are simpler than open-ended generation — the model just needs to read and summarize provided passages).

We serve it via Ollama, which is the simplest way to run open-source models locally — one terminal command downloads and serves the model with an OpenAI-compatible API. The trade-off vs GPT-4 is answer quality, but for privacy and cost (free, forever), `phi4-mini` is the right choice here.

QUESTION 09

What happens in the system when Qdrant finds no relevant results?

There are two fallback layers. First, if the combined filter (`doc_id` + `language`) returns no results, `retrieve_chunks()` logs a warning and retries with **only the `doc_id` filter** — dropping the language

constraint. This handles the case where a Hindi question is asked about an English document.

Second, if that also returns nothing (which would mean Qdrant has no chunks for that file at all), an empty list is returned. `build_prompt()` handles this gracefully by substituting "No relevant passages were found" as the context block.

The system prompt instructs phi4-mini to respond with a specific message in this case: "I could not find an answer to your question in the provided documents." — so the user gets a clear, honest response rather than a hallucinated answer.

QUESTION 10

How would you scale this system for production use?

Several things would change. The **LLM** would move from phi4-mini on a laptop to a larger model (Llama 3 70B or hosted API) for better answer quality. **Qdrant** would move from local to Qdrant Cloud for persistence and horizontal scaling.

The **ingestion pipeline** would move to a job queue like Celery with Redis, because embedding documents is CPU-intensive and should not block web requests. The **Quart server** would be deployed behind nginx with SSL, and the React build would be served as static files from a CDN.

We would also add **user authentication** so each user has their own document library (separate Qdrant collection per user), **chunk deduplication** to prevent re-uploading the same file from creating duplicate vectors, and **streaming responses** so the AI answer appears word by word rather than all at once.

SECTION 06

Glossary

Every technical term used in this project, defined in plain English.

RAG

Retrieval-Augmented Generation. A technique where an AI model is given relevant document passages as context before generating an answer. Prevents hallucination by grounding responses in real content.

Vector / Embedding

A list of numbers (in this project, 768 numbers) that represents the meaning of a piece of text. Similar meanings produce similar vectors. All-mpnet-base-v2 creates these vectors.

Vector Database

A database optimised for storing and searching vectors by similarity. Qdrant is our vector database. Unlike normal databases that search by exact match, vector databases search by meaning.

Cosine Similarity

A mathematical measure of how similar two vectors are, based on the angle between them. Score of 1.0 = identical meaning, score of 0 = completely unrelated.

Chunking

Splitting a large document into smaller pieces (chunks) before storage. In Talk2Books, chunks are 500 characters with 50-character overlap.

Chunk Overlap

The number of characters shared between adjacent chunks. The 50-character overlap in Talk2Books ensures that sentences at chunk boundaries are not cut off awkwardly.

Qdrant

The open-source vector database used in Talk2Books. Runs locally, stores document chunks as vectors, and retrieves the most similar chunks for any query.

Ollama

A tool for downloading and running open-source AI language models locally. It exposes them via a local HTTP API on port 11434.

phi4-mini

Microsoft's lightweight language model. Used in Talk2Books to generate answers from retrieved document passages. Runs locally.

LLM

Large Language Model. An AI model trained on massive text datasets that can understand and generate human language. phi4-mini is an LLM.

Hallucination

When an AI generates plausible-sounding but factually incorrect information. RAG prevents this by forcing the AI to base answers on retrieved document content.

Quart

A Python async web framework (similar to Flask). Used to build the Talk2Books API server. "Async" means it can handle multiple requests simultaneously without blocking.

CORS

Cross-Origin Resource Sharing. A browser security policy that blocks requests between different ports/domains. Solved in Talk2Books using Vite's built-in proxy.

Metadata

Extra information stored alongside each document chunk in Qdrant. In Talk2Books, this includes: file_name, language, and source path. Used for filtering search results.

FieldCondition

A Qdrant filter that restricts search results to chunks where a specific metadata field matches a value. Used to filter by document name and language.

Sentence Transformer

A type of AI model specifically trained to convert sentences into vectors where similar meanings are close together. all-mpnet-base-v2 is a sentence transformer model.

React

A JavaScript library from Meta for building user interfaces. React breaks UIs into reusable "components". The Talk2Books UI is built entirely with React.

Vite

A modern JavaScript development tool that serves and bundles React applications. Also acts as a proxy to forward API requests to the Python backend.

useState / useCallback

React hooks — functions that let components remember values (useState) and cache functions to avoid unnecessary re-creation (useCallback).

Normalisation (text)

Cleaning raw text before storage: collapsing extra whitespace, fixing Unicode encoding. Done in ingest.py using Python's `unicodedata.normalize()`.

You built a complete, production-quality local AI document assistant from scratch. No cloud. No API keys. Fully private. That is genuinely impressive engineering.